COMP 138 RL: Homework 3: Reinforcement Learning TD Problem

Longji Li

March 31, 2025

1 Introduction

In this assignment, we will explore the efficiency of off-policy prediction algorithms in reinforcement learning, focusing on temporal-difference (TD) methods. This paper presents using TD methods and MC methods to solve the Exercise 7.10 using importance sampling [1].

2 Problem

Following the prompt in Exercise 7.10 of Sutton and Barto's Reinforcement Learning book[1], the goal is to compare the efficiency of two different TD methods using importance sampling (7.1 and 7.9) and weighted (per-step) importance sampling (7.13 and 7.2) in a off-policy prediction problem. In this paper, we use a game "Frozen Lake"[2] (FrozenLake-v1) as the environment.

2.1 Environment

Frozen Lake is small game that the agent need to walk on a slippery and iced surface to reach the goal. In the part 1, we implement a non-slippery version of the FrozenLake-v1 environment from OpenAI Gymnasium. It consists of a 4×4 grid world with a single terminal goal (state 15), which gives a reward of 1 upon reaching it. All other states (frozen: 0 ; hole: 0) give 0 reward.

Figure 1: Frozen Lake Map

3 Related Work

Unlike Monte Carlo methods, n-step methods (7.1) [1] do not require waiting until the end of an episode to perform updates. The n-step return can be computed incrementally and used to update the value function as soon as sufficient steps (n steps) have been observed. This allows for earlier and more frequent updates based on the agent's action choices during the episode. In off-policy n-step TD, the update for time will be weight by the important sampling ratio:

$$\rho_{t:h} \doteq \prod_{k=t}^{\min(h,T-1)} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

, where T is the terminal time.

The off-policy importance sampling methods used in this paper are derived from the textbook[1]. The n-step TD is discussed in Chapter 7 as an algorithm of TD Prediction and the algorithm we implemented in our experiment is shown as the following graph:

n-step TD for estimating $V \approx v_\pi$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take an action according to $\pi(\cdot|S_t)$
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then $T \leftarrow t + 1$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $V(S_\tau) \leftarrow V(S_\tau) + \alpha\,[G - V(S_\tau)]$
  Until $\tau = T - 1$

Figure 2: Off-policy n-step TD

4 Approach

4.1 State/ Action

The environment has 16 states(0 to 15) as it shown in Fig 1, state 0 is the start position, state 5,7,11, and 12 are holes, and state 15 is the destination(goal). Other states are frozen surface. The agent will take an action from the actions space: 0: Move left; 1: Move down; 2: Move right; 3: Move up. The target policy π is a stochastic policy that selects RIGHT with 80% probability and DOWN with 20% probability. The behavior policy μ is a uniform random policy, which means that we randomly select an action from the actions space.

4.2 Setup

According to the Frozen Lake's rules, if the agent reach either a goal or a hole, the run of this episode will end, for example:

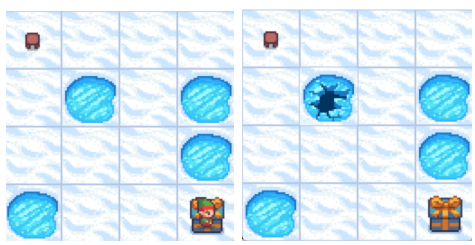

Figure 3: Agent Reached A Goal or Hole

4.3 Experiments

In the first part of the experiment, we applied Monte Carlo control with epsilon-greedy exploration to learn a near-optimal policy from random behavior. This learned policy was then fixed as the target policy π for our off-policy prediction task. We implemented n-step TD with ordinary importance sampling (based on Equations 7.1 and 7.9) and per-decision importance sampling (based on Equation 7.13) to estimate the state-value function v_π. For the MC control, we used 50000 episodes, $\gamma = 0.99$, and $\epsilon = 0.1$. Then, we run two different n-step TD prediction with n = 1, 3, 5, 10.

Our goal was to compare the data efficiency of the two methods in predicting v_π. To evaluate their performance, we computed the mean squared error (MSE) between the estimated value function V and the true V^π obtained via Monte Carlo methods.

5 Result

The result is shown as the following graph:

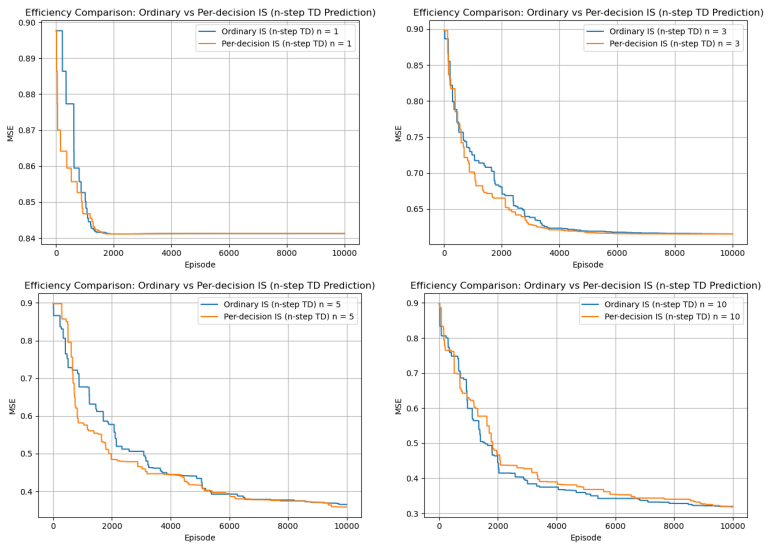

Figure 4: Experiment results with different n

6 Discussion

Based on the results shown in the figure 4, we find that the per-decision importance sampling method (7.13) performs better than the ordinary importance sampling method (Equations 7.1 and 7.9) in terms of data efficiency when $n = 1, 3$, and 5. It achieves lower mean squared error (MSE) faster and exhibits more stable convergence.However, when $n = 10$, the ordinary method appears to converge more quickly. One possible explanation is that, in long episodes, the cumulative importance sampling ratio in the per-decision method tends to decay rapidly (due to multiple steps of off-policy actions), which reduces the effective updates.

Additionally, in shorter episodes (e.g., when the agent run into a hole or a wall), the ordinary method often has a zero importance sampling ratio, leading to no updates, while the per-decision method still use the previous value. This explains its better performance when n is small.

7 Additional Question

We have tried the non-slippery in Frozen Lake, but what if we do a slippery surface? How does the performance of off-policy TD prediction methods change in stochastic environments?

7.1 slippery?

Slipper surface means that the agent may move perpendicular to the intended direction occasionally.

7.2 Experiment

The main part of the experiment remains the same as the part, we just adjust the non-slippery surface to a slippery surface. Similarly, we use the MC control to find a near optimal policy as the target policy. Then we run the two different TD prediction on the new environment, using n = 1, 3, 5, 10 and episodes = 50000.

7.3 Result / Discussion

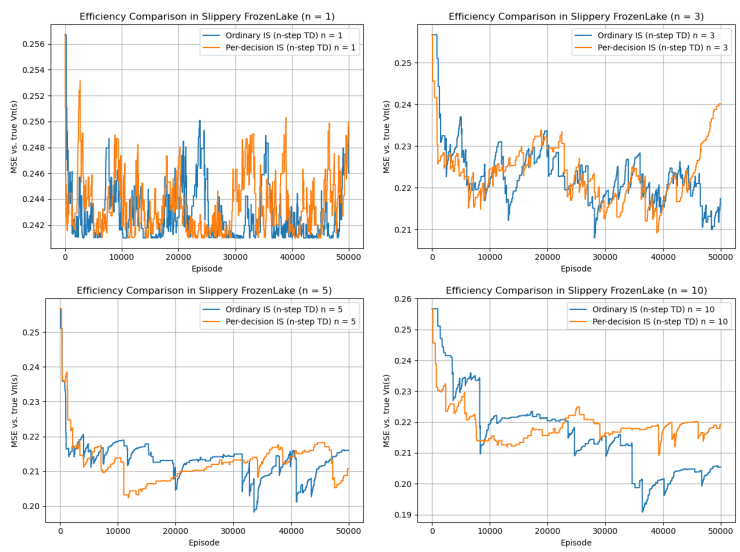

Figure 5: Experiment results when using slippery environment

In the slippery Frozen Lake environment, we find that it has significant differences in the performance of ordinary and per-decision importance sampling methods across different n-step values. For small n (e.g., n=1), both methods

show high variance and poor convergence due to the stochastic moving. As n increases, convergence improves. However, while per-decision IS is slightly better for shorter n, its performance deteriorates with larger n due to the compounding decay of stepwise importance ratios. In contrast, ordinary IS benefits more from longer trajectories and eventually better than per-decision IS at n=10.

8 Conclusion

In this assignment, we explored the efficiency of off-policy n-step temporal-difference (TD) prediction methods using importance sampling in both deterministic (non-slippery) and stochastic environments (slippery). We compared the ordinary importance sampling method (Equations 7.1 and 7.9) and the per-decision importance sampling method (Equation 7.13) in the context of the FrozenLake-v1 environment. The results in the non-slippery environment showed that per-decision method performs better when n is small (e.g., $n = 1, 3, 5$) due to its lower variance and small updates even in shorter episodes. However, ordinary TD method converged faster when $n = 10$, as it applies a single importance ratio over the entire return.

Moreover, to explore the methods in stochastic env, we changed our experiment to the slippery version of the environment using stochastic transitions. The results illustrated that both methods experienced higher variance even we used small n. Thus, we cannot say which method has a better performance. In order to explore this session, we still need to conduct more experiments.

Overall, our experiments showed off-policy TD algorithm using (7.13) and (7.2) is generally more efficient for small n, while ordinary method (7.9, 7.2) becomes slightly more effective with larger n in both stochastic and deterministic environment.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Bradford Book, Cambridge, MA, second edition, 2018.

[2] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.